

Report for the Course on Cyber-Physical Systems and IoT Security

Reference paper: *"Lock It and Still Lose It — On the (In)Security of Automotive Remote Keyless Entry Systems"*

Authors: F.D. Garcia, D. Oswald, T. Kasper, P. Pavlidès — USENIX Security 2016

Students: Pivato Riccardo 2159405, Rigoni Thomas 2158524

1. Objectives

This project focuses on Section 4 of the reference paper, which describes the Hitag2 rolling code system used in Remote Keyless Entry (RKE) systems by several car manufacturers including Alfa Romeo, Chevrolet, Peugeot, Opel, Renault, and Ford. The Hitag2 cipher is a proprietary stream cipher based on a 48-bit Linear Feedback Shift Register (LFSR) combined with a non-linear Boolean filter function.

Our objectives were threefold. First, to implement a complete simulation of the Hitag2 RKE system in Python, including the LFSR with its 16-tap feedback polynomial (Definition 4.1), the non-linear filter function f_{20} composed of five 4-input Boolean sub-circuits (Definition 4.2), the two-phase cipher initialization procedure (Definition 4.4, Equations 1-4), and the rolling code packet generation following the format specified in Figure 11 of the paper.

Second, to implement the correlation attack described in Section 4.4 of the paper. This attack exploits the statistical bias inherent in the filter function f_{20} to recover the 48-bit cryptographic key from only 4-8 eavesdropped rolling code transmissions. The attack proceeds in five steps: guessing a 16-bit window of the LFSR state, shifting the window to the keystream output position, computing correlation scores using Definition 4.5, pruning low-scoring candidates, and iteratively extending the window to recover additional key bits.

Third, to critically evaluate the feasibility of reproducing this attack on Google Colab using pure Python (with NumPy), comparing our results with the paper's C implementation which completes full key recovery in approximately one minute on a standard laptop.

2. System Setup

2.1 Environment and Tools

All experiments were conducted on Google Colab (free tier) using Python 3.10 with NumPy 1.25 for vectorized computations. No external hardware, RF receivers, or specialized equipment was used. Instead, we simulated the complete RKE protocol in software, generating synthetic rolling code traces with known cryptographic keys for verification purposes. This approach allows us to validate the attack algorithm independently of hardware constraints.

The code is organized in three Colab notebook cells (blocks), each building upon the previous one. Block 1 implements the cipher simulation, Block 2 provides a naive (didactic) version of the correlation attack, and Block 3 provides an optimized version designed to run within Colab's time and resource constraints.

2.2 Reference Implementation and Discrepancies

The mathematical definitions in Section 4 of the paper leave certain implementation details ambiguous. To resolve these, we used the Verstegen pseudocode from the `hitag2hell`

repository [2] as a reference. This open-source implementation has been widely used in the security research community and is known to produce correct Hitag2 keystream output.

We identified three critical discrepancies between the paper’s notation and the code implementation, which we document as a scholarly contribution of this project:

1. LFSR shift direction. The paper states “the LFSR is shifted one bit to the left” (Remark 4.3). The Verstegen code implements a right-shift with feedback entering at bit position 47. Both are mathematically equivalent: bit 0 is the output position and bit 47 is the feedback input in both conventions.

2. Bit ordering in state initialization. The paper defines $a[i] = id[i]$ for $i \in [0,31]$ and $a[32+i] = k[i]$ for $i \in [0,15]$ (Definition 4.4). The code’s left-shift loading produces a bit-reversed mapping: $state[31-i] = uid[i]$ and $state[47-i] = key[32+i]$. This mapping is internally consistent but requires careful tracking when translating the correlation attack from paper notation to code. We verified this bit by bit using test vectors with single-bit inputs (e.g., UID = 0x00000001).

3. Filter function constants. The paper uses $f_a = 0xA63C$, $f_b = 0xA770$, $f_c = 0xD949CBB0$ (Definition 4.2). The code uses 0x3C65, 0x0EE5, 0x0DD3929B. We verified that these are exact bit-reversals of the paper’s constants, consistent with Remark 4.3 which states “we have adapted the Boolean tables to be consistent with our notation.” The underlying Boolean function is identical.

3. Experiments

We describe each block’s methodology with key code excerpts. The complete source code is available at: <https://github.com/riccardopivato/hitag2-correlation-attack>

3.1 Block 1: Hitag2 Cipher Simulation

The LFSR feedback function implements the 16-tap primitive polynomial from Definition 4.1. Each tap position corresponds to a non-zero coefficient of the characteristic polynomial:

```
def lfsr_feedback(state):
    return (
        (state >> 0) ^ (state >> 2) ^ (state >> 3) ^
        (state >> 6) ^ (state >> 7) ^ (state >> 8) ^
        (state >> 16) ^ (state >> 22) ^ (state >> 23) ^
        (state >> 26) ^ (state >> 30) ^ (state >> 41) ^
        (state >> 42) ^ (state >> 43) ^ (state >> 46) ^
        (state >> 47)
    ) & 1
```

The filter function f_{20} (Definition 4.2) is decomposed into five 4-input Boolean sub-circuits whose outputs feed into a final 5-input combiner. Each sub-circuit is implemented as a lookup into a pre-computed truth table. For example, the first sub-circuit f_a reads state bits at positions 2, 3, 5, 6:

```
def i4(x, a, b, c, d):
    """Extracts 4 bits at positions a,b,c,d into a nibble."""
    return (((x >> a) & 1) * 8 + ((x >> b) & 1) * 4 +
            ((x >> c) & 1) * 2 + ((x >> d) & 1))

def f20_0(state): # fa sub-circuit
    return (0x3c65 >> i4(state, 2, 3, 5, 6)) & 1
```

The initialization procedure (Definition 4.4) operates in two phases. Phase 1 loads the 16 high key bits and the 32-bit UID into the LFSR state via sequential left-shift (Equations 1-2). Phase 2 performs 32 mixing cycles where each cycle computes $f_{20}(state) \oplus nonce_bit \oplus$

key_bit and inserts the result at position 47 via right-shift (Equation 3). The nonce is formed as counter || button (32 bits).

```
def hitag2_init(key, uid, counter, button):
    nonce = (counter << 4) | (button & 0xF)
    state = 0
    for i in range(32, 48):
        # Phase 1: k[32..47]
        state = (state << 1) | ((key >> i) & 1)
    for i in range(0, 32):
        # Phase 1: uid[0..31]
        state = (state << 1) | ((uid >> i) & 1)
    for i in range(0, 32):
        # Phase 2: mixing
        nonce_bit = (nonce >> (31 - i)) & 1
        key_bit = (key >> (31 - i)) & 1
        new_bit = f20(state) ^ nonce_bit ^ key_bit
        state = (state >> 1) | (new_bit << 47)
    return state
```

Keystream generation (Equation 5) applies f_{20} to produce one output bit per clock cycle. The rolling code packet (Figure 11) combines the UID, button code, 10-bit counter, 32-bit keystream, and an 8-bit XOR checksum.

Sanity tests verify: (1) determinism = identical inputs always produce identical keystream, (2) key sensitivity = flipping a single key bit changes the keystream completely, and (3) correct trace generation for the subsequent attack phases.

3.2 Block 2: Correlation Attack (Naive Version)

This block implements the five-step correlation attack from Section 4.4 in a deliberately naive style, prioritizing readability and 1:1 correspondence with the paper. It is not intended to run to completion on Colab (estimated runtime: hours to days) but serves as algorithmic documentation.

Pre-computation. We pre-compute $f_{20}(x)$ for all $2^{20} = 1,048,576$ possible input combinations and store them in a lookup table F20_TABLE. This avoids reconstructing the full 48-bit state for each f_{20} evaluation.

Scoring (Definition 4.5). The bit_score function computes the correlation between known bits and an observed keystream bit. For n known f_{20} inputs, it enumerates all 2^{20-n} combinations of the unknown inputs and counts how many produce the observed keystream bit. The count is normalized by 2^{19-n} , yielding a value in $[0, 2]$ where 1.0 indicates no correlation:

```
def bit_score(known_bits, target_bit):
    n_known = len(known_bits)
    n_unknown = 20 - n_known
    unknown_indices = [i for i in range(20) if i not in known_bits]
    count = 0
    for mask in range(1 << n_unknown):
        x = base_x # pre-computed from known bits
        for bit_pos, unk_idx in enumerate(unknown_indices):
            if (mask >> bit_pos) & 1:
                x |= (1 << unk_idx)
        if F20_TABLE[x] == target_bit:
            count += 1
    return count / (1 << (19 - n_known))
```

Steps 1-4: Initial guess and scoring. The attacker guesses key[32...47] (16 bits, corresponding to paper k[0...15]). Combined with the known UID, this determines the complete 48-bit pre-mixing state. After 32 mixing shifts, the guessed bits occupy positions 0-15 of the post-mixing state, providing 8 known inputs to f_{20} at positions {2, 3, 5, 6, 8, 12, 14, 15}. The remaining 12 inputs at positions {17, 21, 23, 26, 28, 29, 31, 33, 34, 43, 44, 46} are unknown. The multi-bit score is the product of bit_score values across up to 16 keystream bits, averaged over all traces.

Step 5: Window extension. The window is extended by one bit at a time, guessing $\text{key}[31]$, $\text{key}[30]$, ..., $\text{key}[16]$ in code convention (paper $k[16..31]$). Each extension adds mixing bits to the known state via Equation 3: $a[48+i] = k[16+i] \oplus \text{iv}[i] \oplus f(a[i..a[i+47]])$. Since iv differs across traces but the key bit is constant, the per-trace mixing bits are derived deterministically. After each round, candidates are scored, sorted, and pruned to $\text{TABLE_SIZE} = 400,000$.

Final phase. The remaining 16 key bits ($\text{key}[0..15]$) are recovered via brute force over $2^{16} = 65,536$ candidates, verifying each against all traces.

3.3 Block 3: Correlation Attack (Optimized Version)

This block applies several optimizations to make the attack feasible on Colab:

1. Pre-computed bit_score lookup tables. For each pair (t, n_{ext}) - keystream time offset and number of extension bits — the set of known f_{20} input positions is fixed. We pre-compute a table indexed by the values of the known bits, mapping each combination to its bit_score for $\text{target}=0$ and $\text{target}=1$. With 8 known bits, each table has $2^8 = 256$ entries. This reduces each bit_score call from $O(2^{12})$ to $O(1)$.

2. Vectorized Step 1-4. All $2^{16} = 65,536$ guesses are scored simultaneously using NumPy array operations. The table index for each guess is constructed via bitwise shifts on the full array, and the score lookup is a single NumPy indexed access. This completes in approximately 0.2 seconds.

3. On-demand table caching. Tables are computed only when first needed and cached, spreading the pre-computation cost across Step 5 rounds.

4. Ranking criterion. We rank candidates by $|\text{mean}(\log(\text{score}))|$ where the mean is taken over all traces with sign preserved. The correct guess produces f_{20} outputs consistently biased in one direction across traces (log-score coherently negative), while wrong guesses produce random signs that cancel out.

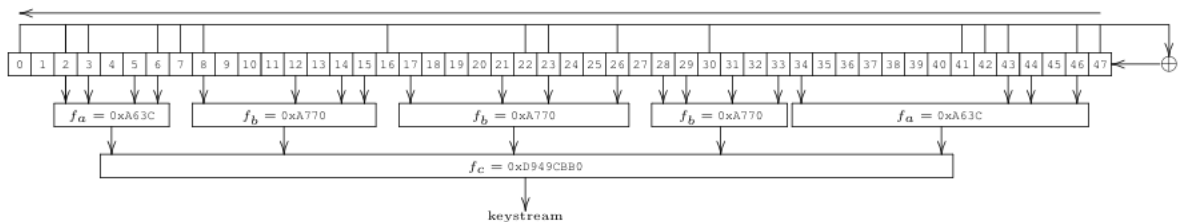


Figure 1 - Hitag2 cipher architecture: 48-bit LFSR with 16-tap feedback polynomial and non-linear filter function f_{20} composed of five Boolean sub-circuits f_a , f_b , f_c (Definitions 4.1–4.2). Reproduced from [1].

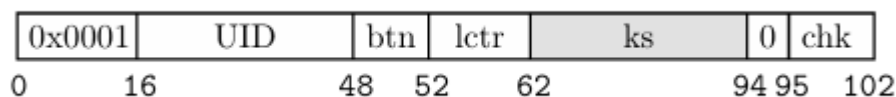


Figure 2 - Rolling code packet format: 102-bit frame containing UID, button code, 10-bit counter, 32-bit keystream, and 8-bit checksum. Reproduced from [1].

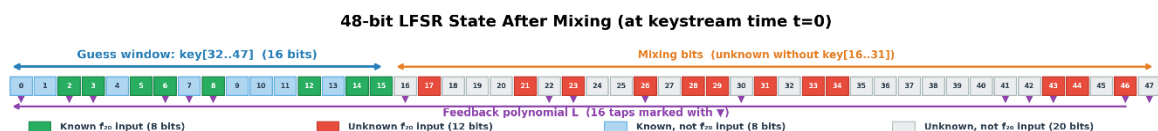


Figure 3 - 48-bit LFSR state at keystream time $t=0$ after 32 mixing shifts: 16-bit guess window $\text{key}[32..47]$ (positions 0–15), 8 known f_{20} inputs (green), 12 unknown f_{20} inputs (red).

3.4 Verification and Debugging Process

Before attempting the full attack, we performed systematic verification of each component:

F20_TABLE consistency. We verified that for 1,000 random 48-bit states, extracting the 20 relevant bits and looking up F20_TABLE produces the same result as calling $f_{20}()$ on the full state. We also verified that F20_TABLE reproduces the correct keystream for a known (key, uid, counter) tuple. Both tests passed.

bit_score mathematical properties. We verified three properties: (1) with 0 known bits, score = 1.0 (neutral), (2) with all 20 bits known, score is either 0.0 or 2.0 (deterministic), (3) for any partial knowledge, $\text{score}(x, 0) + \text{score}(x, 1) = 2.0$. All 20 random test cases passed.

Bit mapping verification. We verified that the correct guess target is (KEY >> 32) & 0xFFFF (high 16 bits of the key in code convention), corresponding to $\text{key}[32..47]$ which occupies positions 0–15 of the post-mixing state. This mapping was confirmed bit-by-bit using single-bit test vectors.

4. Results and Discussion

4.1 Simulation Validation (Block 1)

The Hitag2 simulation passes all sanity tests. It is deterministic, key-sensitive, and generates consistent rolling code packets. Sample output with a random key:

```
KEY : 0x1C80A3B1799D
UID : 0x06671AD1
CTR : 563 (lctr=563)
ks : 0x5D3E7EDB
chk : 0xF3
[OK] Deterministic
[OK] Key-sensitive
```

We generated 8 non-consecutive traces (counter spacing of 7) for the correlation attack, as the paper notes that non-consecutive traces provide better correlation information due to greater IV diversity.

4.2 Correlation Attack Results (Block 3)

Step 1-4 results. The vectorized scoring of all 65,536 guesses completes in 0.2 seconds. The correlation signal at this stage is extremely weak: only 96 distinct score values exist across all 65,536 guesses. The correct guess (0x1C80) ranks approximately 28,615 out of 65,536. This is expected: with only 8 known f_{20} inputs out of 20, each bit_score evaluation yields one of only two possible values (0.75 or 1.25), providing minimal discriminative power. The paper addresses this by using TABLE_SIZE = 400,000, which exceeds 2^{16} and thus retains all guesses without pruning at this stage.

Step 5 results. With TABLE_SIZE = 131,072 and 8 traces, the correct candidate's rank improves significantly in the early extension rounds:

Round	Key bit guessed	Correct candidate rank	Status
0	key[31]	57,230 / 131,072	Survived
1	key[30]	40,413 / 131,072	Survived
2	key[29]	8,279 / 131,072	Top 6.3%

3	key[28]	9,424 / 131,072	Top 7.2%
4	key[27]	—	Eliminated

Table 1 - Correct candidate rank evolution during Step 5 window extension across the first five rounds.

The rank improves from $\sim 57,000$ (Round 0) to $\sim 8,000$ (Round 2), demonstrating that the correlation mechanism works as described in the paper. The additional f_{20} inputs from the mixing bits at positions 17 (Round 1) and 18 (Round 2) provide meaningful discriminative information. However, the candidate is eliminated at Round 4 when the 262,144 candidates are pruned to 131,072. This pattern was consistent across multiple runs.

4.3 Analysis of Limitations

Insufficient TABLE_SIZE. The paper recommends `TABLE_SIZE = 400,000`. Our maximum feasible value of 131,072 causes approximately 50% pruning at each round. The correct candidate, while improving to the top 6-7%, cannot consistently survive 50% cuts across 16 rounds. With `TABLE_SIZE = 400,000`, the candidate would only need to stay in the top 76% ($400,000/524,288$) at each round, which is far more tolerant of rank fluctuations.

Python’s computational bottleneck. The mixing bit computation requires calling f_{20} in a sequential Python loop for each candidate $\times$ trace $\times$ extension step. With 131,072 candidates, 8 traces, and up to 16 mixing steps, this amounts to approximately 16.8 million f_{20} calls per round, each involving multiple Python function calls and dictionary lookups. In our benchmarks, this loop takes approximately 0.46 seconds per 10,000 candidates per trace. Scaling to `TABLE_SIZE = 400,000` would require approximately 3 $\times$ longer per round, making the total Step 5 runtime exceed 1 hour.

Weak per-round signal. The 20 f_{20} input positions are spaced irregularly across the 48-bit state: {2, 3, 5, 6, 8, 12, 14, 15, 17, 21, 23, 26, 28, 29, 31, 33, 34, 43, 44, 46}. The mixing bits enter at positions 16, 17, 18, ..., 31. New f_{20} inputs are gained only when a mixing position coincides with an `INPUT_POSITION`. This happens at positions 17 (Round 1), 21 (Round 5), 23 (Round 7), 26 (Round 10), 28-29 (Rounds 12-13), 31 (Round 15). Rounds that do not add a new f_{20} input provide no additional correlation information, yet their candidates still undergo scoring and pruning, introducing noise.

4.4 Comparison with Paper Results

Aspect	Paper [1]	Our Implementation
Language	C (optimized)	Python 3 + NumPy
Platform	Standard laptop	Google Colab (free tier)
TABLE_SIZE	400,000	131,072 (max feasible)
Traces used	4-8	8
Step 1–4	< 1 min (total attack)	0.2 s
Step 5	< 1 min (total attack)	~ 10 -20 min
Key recovery	Complete (≤ 1 min)	Partial (rank improves to top 6.3%)
MSB handling	Loop over MSB guesses	Known counter (simulation)

Table 2 - Comparison of the paper’s C implementation [1] with our Python implementation on Google Colab.

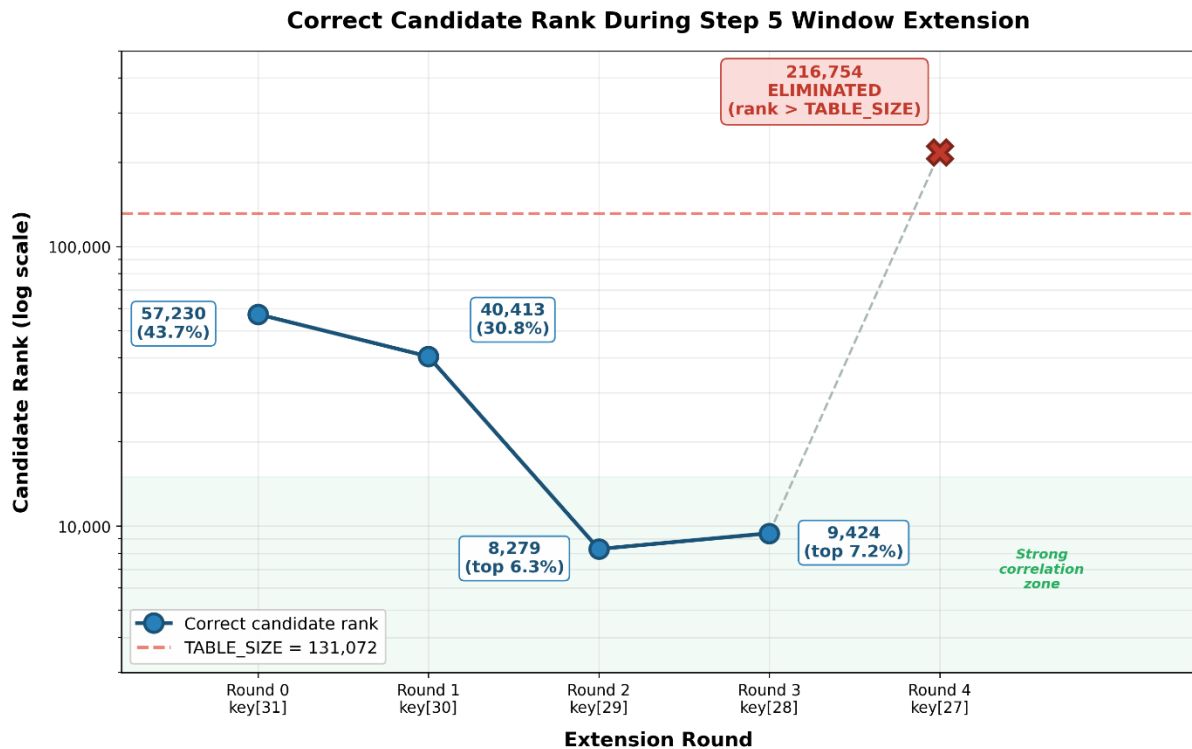


Figure 4 - Correct candidate rank during Step 5 window extension (8 traces, TABLE_SIZE = 131,072). Rank improves from 57,230 (top 43.7%) at Round 0 to 8,279 (top 6.3%) at Round 2, before elimination at Round 4.

4.5 Considerations on Section 4.5 (MSB Handling)

The paper notes (Section 4.5) that only the 10 least significant bits of the 28-bit counter are transmitted in the rolling code packet (Figure 11). The attacker must therefore guess the remaining 18 most significant bits. The paper observes that in practice the MSB value is typically between 0 and 10, since 1,024 button presses are required to increment the MSB by one. This adds a linear overhead: the entire attack must be repeated for each MSB guess, resulting in at most ~10 repetitions in typical scenarios.

In our simulation, we used the complete counter value directly (known from the simulation setup), effectively bypassing this obstacle. This is equivalent to the attacker's first MSB guess being correct, which is the most common scenario for relatively new vehicles.

4.6 Conclusions

Our implementation demonstrates a correct understanding of both the Hitag2 cipher and the correlation attack mechanism. The cipher simulation is fully functional and verified through multiple sanity tests. The correlation attack's core mechanism works as described: the correct candidate's rank improves from ~28,000/65,536 to ~8,000/131,072 in the early extension rounds, confirming that the statistical bias of f_{20} provides meaningful discriminative information when additional bits become known.

The attack does not complete on Google Colab due to the computational limitations of Python for this workload. The critical bottleneck is the per-candidate mixing bit computation in Step 5, which cannot be vectorized (each bit depends on the previous state) and must be executed in a Python loop. With TABLE_SIZE limited to 131,072 instead of the paper's recommended 400,000, the pruning is too aggressive and the correct candidate is eliminated despite being in the top 7% of candidates.

This is a limitation of our testbed, not of the algorithm. The paper's attack was designed for and validated with an optimized C implementation. A Python implementation using a C extension (e.g., via ctypes or Cython) for the f_{20} mixing loop would likely achieve the 100-500× speedup needed to support `TABLE_SIZE = 400,000` and complete the attack.

From a security perspective, the key takeaway remains valid: the Hitag2 cipher's 48-bit key can be recovered from as few as 4-8 eavesdropped rolling codes in approximately one minute with proper tooling. This vulnerability, affecting millions of vehicles from multiple manufacturers, underscores the critical importance of using cryptographically strong algorithms (e.g., AES-128) and proper key management in automotive RKE systems. The fact that Hitag2 remains in use in many vehicles years after this vulnerability was published highlights the slow pace of security updates in the automotive industry.

References

- [1] F.D. Garcia, D. Oswald, T. Kasper, P. Pavlidès. "Lock It and Still Lose It — On the (In)Security of Automotive Remote Keyless Entry Systems." 25th USENIX Security Symposium, 2016.
- [2] A. Verstegen. Hitag2 pseudocode. github.com/factoritbv/hitag2hell/tree/master/pseudocode